VulGuard : Vulnerability Detection and Resolution Tool

CSC-300

Lab Based Project Report

Aadit Kumar Sahoo 23114001
Utkarsh Kumar 23114101

Department of Computer Science and Engineering
Indian Institute of Technology Roorkee
Roorkee - 247667 (INDIA)

Supervisor : Dr. Sandeep Kumar Garg

May, 2026

INDIAN INSTITUTE OF TECHNOLOGY
ROORKEE, ROORKEE

Candidate's Declaration

We declare that the work carried out in this report entitled **"VulGuard : Vulnerability Detection and Resolution Tool"** is presented on behalf of the fulfillment of the course CSC-300 submitted to the **Department of Computer Science and Engineering, Indian Institute of Technology, Roorkee** under the supervision and guidance of **Prof. Sandeep Kumar Garg, Department of Computer Science and Engineering**

Aadit Kumar Sahoo 23114001
Utkarsh Kumar 23114101

Date: 13 May 2026

Supervisor
(Dr. Sandeep Kumar Garg)

Abstract

The process of detecting software vulnerabilities requires execution in early stages because this process leads to better security and software system reliability. The conventional methods for detecting vulnerabilities face difficulties when trying to find insecure code while they fail to deliver useful instructions for developers. This report presents a vulnerability detection framework for C/C++ source code that combines deep learning techniques with Large Language Models to support automated vulnerability detection, explanation, and fix suggestion generation.

The system combines machine learning code analysis with a retrieval-augmented generation RAG system that uses vulnerability knowledge sources to enhance contextual understanding and detection performance. The framework enables developers to receive security recommendations while conducting real-time analysis work. The testing used actual C/C++ code samples which showed better vulnerability detection results and successful identification of vulnerable code areas.

Acknowledgements

We would like to sincerely thank Prof. Sandeep Garg from the Department of Computer Science and Engineering, IIT Roorkee, for giving us the opportunity to work on this project. His continuous support, guidance, and encouragement have been invaluable to us throughout the process.

A big thank you to all the team members for their hard work and collaboration. This project wouldn't have been possible without each one of you.

Aadit Kumar Sahoo 23114001
Utkarsh Kumar 23114101

Date: 13 May 2026

Contents

Chapter 1

Introduction

Modern application security experiences a major security risk because software vulnerabilities exist. Security assurance becomes an ongoing security challenge when software systems become more complex and operate in various environments and use multiple third-party components. Supply chains enable vulnerabilities to spread beyond their initial defects because they now function as security threats to all linked systems. The organization requires automated vulnerability detection to protect their systems from security threats while keeping their repair expenses at a minimum.

Secure software development practices face limitations because organizations operate under constraints related to time requirements and development team size and software evaluation costs. Organizations with large codebases and fast release schedules need automated auditing systems because their manual auditing processes cannot handle their operational demands. Static analysis tools from traditional methods offer extensive codebase examination abilities but their results produce numerous false alerts because the tools lack proper understanding of code context. This results in alert fatigue which leads to ineffective prioritization and prolonged time needed for remediation processes. Dynamic analysis and fuzzing methods produce successful outcomes but their execution needs both runtime environments and detailed setup work and substantial computing power which prevents their application to all system modifications.

Learning-based approaches use code semantic and structural elements to identify potential security vulnerabilities. Research prototypes function as state-of-the-art models because they need custom operating systems to work and produce results that users find complex to understand. The current system creates a barrier which prevents users from accessing all the functions of the model. To work successfully detection requires speed together with clear explanations and detection systems which need to work within developers existing tools that enable them to work efficiently between different tasks while delivering practical assistance.

VulGuard solves these problems through its implementation of model-based vulnerability detection which operates within Visual Studio Code. The extension extracts source code

from the current editor screen and transmits it to a continuous inference system which produces instant results. The system shows unsafe code sections together with risk summaries and offers LLM-generated solutions which use retrieval-based contextual information. The design enables users to perform quick assessments while they conduct their ongoing work in the Integrated Development Environment. VulGuard establishes a link between research-level models and practical software engineering through its user-friendly design and its ability to conduct complex evaluations.

Modern development workflows need both traceability and explainability during their execution. Critical systems require teams to learn which code triggered an alert while understanding how the recommended solution will modify system functionality. Security engineers need more than pure classification outputs when they must explain risk to their stakeholders or demonstrate that their solution will not create new problems. VulGuard requires developers to examine and approve patch candidates which the system presents through its line-by-line context explanations and validated patch.

Real-world code presents another challenge because of its diverse coding patterns. Software projects combine traditional code with current development frameworks while different programming methods and rapid development processes. The system needs to develop patterns which match specific project requirements while maintaining its ability to work across different projects. VulGuard achieves its goal of delivering stable low-latency results through its use of model inference and retrieval-augmented context while maintaining warm inference servers for quick response times to different team and codebase workload sizes.

The security tools need to become part of current security procedures instead of functioning as complete replacements. Development teams depend on three essential tools which include IDEs and code review and continuous integration pipelines. VulGuard provides lightweight on-demand analysis to integrate with existing workflows which allows developers to perform quick triage during development while decreasing the time needed to resolve issues after discovery.

Chapter 2

Initial Work

The initial work focused on understanding how to assign a vulnerability score to code generated by LLMs. We studied how different scoring methods link model outputs to security indicators. The research showed that a scoring system can transform basic prediction results into specific development recommendations for developers.

From [10], we studied quantitative scoring systems such as CVSS, VRSS, and WIVSS and the limitations caused by fixed Impact:Exploitability ratios. The paper highlights that fixed ratios can undervalue high-impact but hard-to-exploit flaws or overvalue easy-to-exploit but low-impact issues. The hybrid model (VIEWSS) uses qualitative ratings to determine which weight should be applied to Impact:Exploitability. The system maintains CVSS-style exploitability inputs while it uses weighted CIA impact to assess confidentiality more heavily before the system determines which 6:4, 5:5, or 4:6 weight distribution to use for impact and exploitability ratings.

The research used CVSS, SSVC, EPSS, and the Exploitability Index to assess real-world Patch Tuesday disclosures according to evidence from [11]. The study shows that these scoring systems often disagree, with weak correlations and low agreement across ranking and categorical bins. The research found that bin-based triage creates analyst overload when excessive vulnerabilities enter top bins while time-based exploit prediction fails to predict exploitation before public catalog confirmation according to EPSS. We decided to treat scoring as a flexible multi-signal system because of our research findings and we need to show developers and reviewers our score calculation process.

The exploratory phase showed strong potential for the concept so we want to develop it into future work. The project needs a dedicated long-term framework for security evaluation and benchmark calibration and cross-model analysis to achieve accurate results and practical applicability. The idea has been preserved as future work for subsequent research and development.

Chapter 3

Related Work

The field of machine learning for software security has experienced many new developments during the past few years. The following works are directly relevant to VulGuard and motivate our design choices.

3.1 VulExplainer

VulExplainer presents a transformer-based hierarchical distillation method which produces human-readable descriptions of security vulnerabilities [8]. The project developed developer-friendly reporting systems which handle explanation quality as their primary requirement. The authors code was tested by us on our local system and we found that their explanation generation worked for test cases which confirms the claims of model.

3.2 VulRepair

VulRepair presents a sequence-to-sequence repair task for vulnerability fixing which uses T5-based models as its base structure [9]. The research shows that it is possible to produce automated repairs from vulnerable code while generating security patches through an automated process. We established the open-source implementation in our local environment and tested its functionality by verifying multiple repair results against their documented performance.

3.3 VulDeePecker

VulDeePecker presents a software vulnerability detection system which uses deep learning methods to analyze code gadgets [2]. The research showed that using neural representations helps detect software vulnerabilities while maintaining the need for high-quality training data to achieve effective results. The gadget-based framing method provides a way to separate vulnerable contexts from larger functional areas.

3.4 GRACE

The research presents GRACE as a system which utilizes graph-based learning together with context modeling to achieve better detection results [7]. The research demonstrates that context fusion enables better classification results through the integration of multiple contextual elements which surpasses the detection capabilities of local historical data.

3.5 Russell et al.

The research study conducted by Russell et al. examines deep learning techniques which enable detection of security vulnerabilities at the function level [3]. The research establishes that automated feature extraction techniques deliver better results than traditional manual rule-based systems for real-world programming environments which contain various types of noise. The research highlights the requirement for developing software representations which maintain their effectiveness across different programming languages and coding practices.

3.6 SySeVR

The system SySeVR develops a representation system which uses syntax patterns and semantic understanding to identify security vulnerabilities throughout software systems [4]. The system enhances its ability to identify vulnerable software elements by uniting syntax patterns with their corresponding semantic relationships. Our research focuses on context-aware signals because we need information which goes beyond basic word recognition signals.

3.7 REVEAL

REVEAL uses graph-based neural networks to analyze code property graphs for finding security weaknesses in software programs, according to research documented in [5]. Our graph-based methodology guides us to examine structural elements which help us identify patterns while we explain the system, which includes analyzing complex data and control flow dependencies that cannot be expressed through standard token patterns.

3.8 Devign

Devign represents code through graph structures and uses gated graph neural networks to perform detection tasks, according to research in [6]. The system shows that graph-based

representations effectively capture control and data flow while structured program analysis enhances learned embeddings.

3.9 LineVul

LineVul enables users to find security weaknesses at the level of individual program lines, according to research in [1]. The method directly supports our strategy of highlighting specific lines while delivering useful results that editors can use. The system enables accurate location detection which drives the need for exact patches that developers can easily understand instead of broad function-based alerts.

Chapter 4

Motivation

The primary obstacle which developers face when designing secure software systems consists of their need to produce precise security vulnerability assessments which can be used in authentic software development environments. Although previous studies created effective detection systems, they still contain multiple limitations which hinder their actual implementation. The existing gaps between current systems and effective systems drive our research to develop a system which offers better usability and greater dependability throughout its complete functionality.

- **Limitations of traditional approaches:** Many tools depend on lexical patterns or shallow features (e.g., bag-of-words or heuristic rules) that miss semantic and contextual cues. The system fails to identify security threats because it overlooks hidden control-flow and data-flow problems while it generates false alerts when secure code matches established patterns.
- **Noisy, heterogeneous codebases:** Actual software development projects combine old system components with different programming styles and multiple software components. The detection performance in live environments decreases because models trained with controlled datasets cannot deal with these particular situations.
- **Fragmented workflows:** Research prototypes frequently run outside the IDE, forcing developers to context switch, export code, and interpret raw logs without guidance. The existing barriers prevent users from implementing the system while they need more time to handle security issues.
- **Coarse-grained outputs:** Function-level alerts without line localization slow down the process of fixing issues and make code review more challenging. Developers need to conduct manual investigations to determine the actual root cause, which results in longer timeframes to resolve problems.

- **Weak explanations and fixes:** Many systems provide scores without reasoning, and automated fixes can be unsafe without grounded context. A practical tool must explain why a line is vulnerable and propose changes that respect surrounding code semantics.
- **Latency and scalability constraints:** Heavy pipelines or cold-start inference prevent developers from using the system during their iterative development process. Teams need fast, repeatable analysis that scales with file size and team velocity.

Chapter 5

Proposed Method

The VulGuard pipeline for vulnerability detection and explanation and fix generation operates within VS Code. Our method combines three components which include model-backed inference and retrieval-augmented context and a review-first patch workflow. The overall process is organized into the following phases.

5.1 Method Phases

- **Phase 1: Input capture and preprocessing.** The extension establishes a content hash by reading the active editor buffer after transforming line endings to standard format. The revision token safeguards the snapshot which contains evidence.
- **Phase 2: Model inference.** The snapshot is converted into JSON format which the inference server receives. The server conducts tokenization together with model forward passes to produce per-line vulnerability scores and a ranked candidate list.
- **Phase 3: Evidence enrichment.** The system creates a contextual window that surrounds each candidate line when the feature gets activated. Evidence snippets get their proximity scores and relevance scores which help maintain LLM grounding.
- **Phase 4: Explanation and fix generation.** The prompt structure gets constructed through the combination of finding evidence and output schema constraints. The LLM produces explanations together with several patch candidates.
- **Phase 5: Review and application.** The system displays ranked candidates to obtain direct consent from users. The system uses an editor edit to implement the chosen patch.

The system uses bounded steps which have predefined time limits and extension configuration settings to implement its various phases. The system allows incremental phase execution because VulGuard uses cached artifacts from previous runs to access recent findings and evidence windows which reduce system downtime.

5.2 System Architecture and Capabilities

The system has two primary components: the VS Code extension (front end) and a persistent Python inference server (back end). The extension transmits the current C/C++ file to the server when a developer begins an analysis which then executes a line-level vulnerability model to generate specific findings and assessment of risk levels. The editor receives results back through inline highlights which display a condensed risk assessment.

The extension establishes communication with the server through a simple JSON protocol which transmits the file path and content hash. The server validates the request, executes the model, and returns a structured response containing per-line scores, vulnerable line indices, and optional metadata for downstream stages. The system architecture maintains user interface responsiveness because the server executes resource-intensive inference processes.

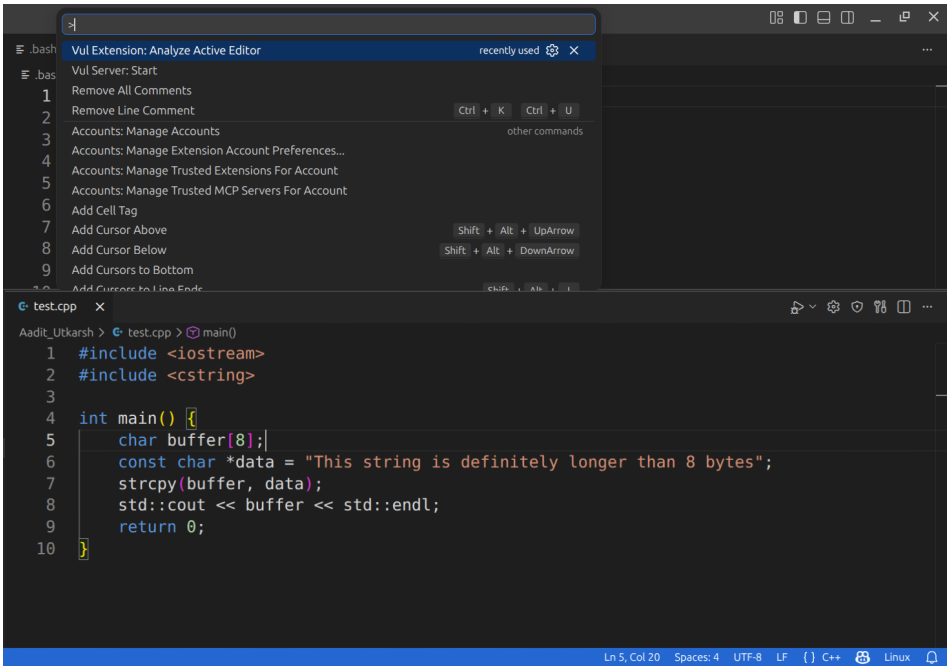

FIGURE 5.1: VulGuard integrated into the VS Code interface for inline vulnerability inspection.

5.3 Retrieval-Augmented Generation and Fix Suggestion

The "Apply Suggested Fix" command by VulGuard allow us to start its fix workflow after finding vulnerabilities. The extension creates a structured prompt when developers choose a particular finding which includes retrieval-augmented context that consists of neighboring lines and evidence snippets and prior findings. The prompt gets delivered to a configured LLM provider which includes local Ollama and a cloud endpoint to produce multiple patch

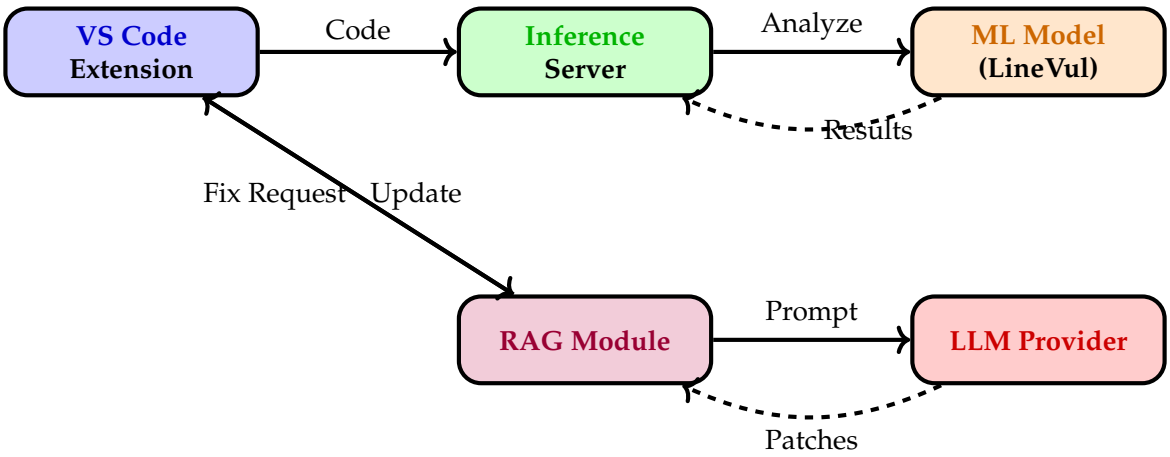

FIGURE 5.2: VulGuard System Architecture: Vulnerability Detection and RAG-based Fix Generation Pipeline

candidates. VulGuard shows the candidates to humans for evaluation and it needs their explicit permission before applying any patch.

The RAG stage creates a restricted context window which covers every vulnerable line while following a user-defined token limit. The system eliminates duplicate evidence snippets and organizes them based on their closeness and importance. This grounding step reduces hallucinations and helps the LLM maintain semantic consistency with the surrounding code. The system uses basic rules to rank patch candidates which include two criteria: minimal diff size and syntax validity before displaying them to users.

5.4 Persistent Server Mode

VulGuard operates its inference system through a persistent server which maintains its system caches at operational status to provide development teams with almost instant feedback. The system design enables users to operate their software throughout their normal integrated development environment work procedures without experiencing any delays. The extension operating in this mode has the ability to automatically activate the inference server while maintaining a single operational process for multiple analysis tasks. The system implementation employs timeouts together with retry mechanisms to stop the editor from becoming unresponsive.

Algorithm 1 End-to-End VulGuard Analysis Pipeline

Require: Active file content C, settings S, timeout T
Ensure: Findings F, patches P, content hash h

$C_{norm} \leftarrow \text{Normalize}(C)$
$h \leftarrow \text{SHA256}(C_{norm})$
$req \leftarrow \{C_{norm}, h, S.mode, S.model\}$
$resp \leftarrow \text{SendToServer}(req, T)$
$scores \leftarrow resp.lineScores$
$F \leftarrow \text{SelectTopK}(scores, S.topK)$
$F \leftarrow \text{ApplyThreshold}(F, S.thresh)$
$F \leftarrow \text{AttachLineContext}(F, C_{norm}, S.window)$
if $S.rag$ **then**
    $E \leftarrow \text{BuildEvidence}(F, C_{norm}, S.ragWindow, S.ragTopK)$
else
    $E \leftarrow \varnothing$
end if
if $S.llm$ **then**
    $P \leftarrow \text{SynthesizePatches}(F, E, S.llm)$
    $P \leftarrow \text{ValidateAndRank}(P, C_{norm})$
else
    $P \leftarrow \varnothing$
end if
return F, P, h

Algorithm 2 RAG-LLM Patch Synthesis

Require: Finding f, code C_{norm}, evidence set E, model settings S
Ensure: Candidate patches P

$W \leftarrow \text{ExtractWindow}(C_{norm}, f.line, S.window)$
$E' \leftarrow \text{RankEvidence}(E, f, S.ragTopK)$
$ctx \leftarrow \text{AssembleContext}(W, f, E', S.budget)$
$schema \leftarrow \text{PatchSchema}()$
$prompt \leftarrow \text{ComposePrompt}(ctx, schema, S.style)$
$raw \leftarrow \text{LLMGenerate}(prompt, S.temperature, S.timeout)$
$P \leftarrow \text{ParsePatches}(raw)$
$P \leftarrow \text{FilterBySyntax}(P, C_{norm})$
$P \leftarrow \text{FilterByDiffSize}(P, S.maxDiff)$
$P \leftarrow \text{RankCandidates}(P, \{\text{minDiff}, \text{contextMatch}\})$
return P

5.5 User Workflow

The following diagram illustrates the detailed steps a user takes to interact with the VulGuard extension during typical development.

The workflow shows its process by using quick feedback cycles which start with detection running for seconds followed by the developer choosing a finding and completing their review of patch candidates before applying them. The system confirms its trustworthiness because it requires explicit user permission before making any code changes. The system provides one-click re-analysis after users apply patches.

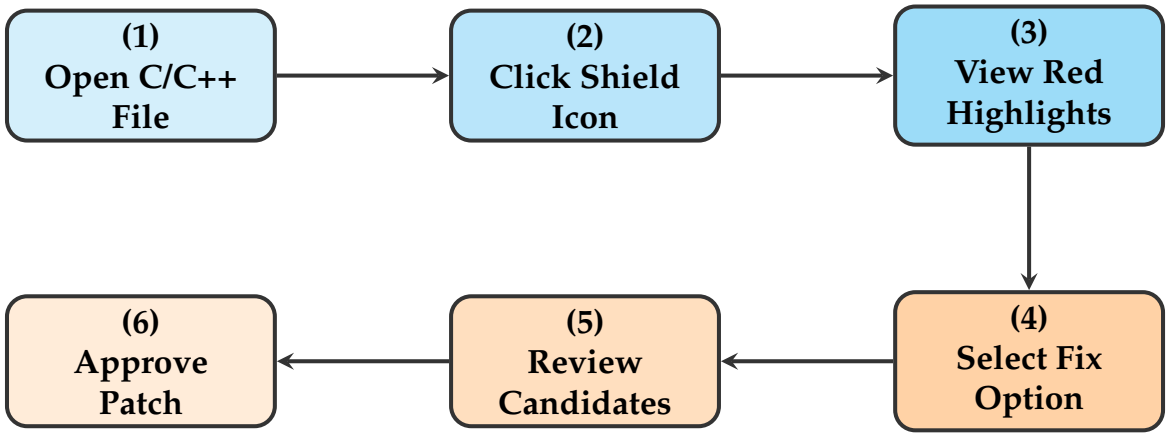

FIGURE 5.3: VulGuard User Workflow: Six-Step Process for Vulnerability Detection and Resolution

Chapter 6

Experiment Evaluation

The chapter assesses VulGuard performance by testing its ability to detect vulnerabilities and provide patching help on two actual datasets. The results include function-level classification performance and line-level tracking results and complete system performance measurements.

6.1 Evaluation Metrics

The analysis of classification results uses established metrics from confusion matrix data. We use Top-*k* localization accuracy to assess line-level localization performance. It determines if a vulnerable line exists among the top-*k* model output lines. We also report line-level F1 computed over the vulnerable line set.

Metric	**Definition**
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$
Precision	$\frac{TP}{TP+FP}$
Recall (pd)	$\frac{TP}{TP+FN}$
False alarm (pf)	$\frac{FP}{FP+TN}$
F1-score	$\frac{2 \cdot Precision \cdot Recall}{Precision+Recall}$
Localization@k	$\frac{1}{\lvert V \rvert} \sum_{v \in V} \mathbb{1}[\text{vuln line in top-k}]$

TABLE 6.1: Evaluation metrics used for VulGuard.

6.2 Datasets and Baselines

The research uses two common datasets for vulnerability assessment which include FFmpeg+Qemu and Big-Vul. We select C/C++ functions which have confirmed vulnerability assessments and we eliminate functions which do not possess accurate line mapping systems

needed for location identification. The baseline methods include VulDeePecker, SySeVR, Devign, Reveal, GRACE and LineVul which utilize different methods for sequence and graph and line-based analysis.

Dataset	**Functions**	**Vuln. %**	**Avg. Lines**	**Files**
FFmpeg+Qemu	21,400	5.8	94	5,200
Big-Vul	34,900	7.2	78	12,600

TABLE 6.2: Dataset statistics after preprocessing and line mapping.

6.3 Comparison with Baselines

Table 6.3 presents results at the function level. VulGuard achieves the highest F1 on both datasets because it outperforms the best baseline GRACE on FFmpeg+Qemu and LineVul on Big-Vul. The system demonstrates better performance because it detects more problems while keeping a good level of precise detection method.

Method	**FFmpeg+Qemu**				**Big-Vul**			
	Acc	Prec	Rec	F1	Acc	Prec	Rec	F1
SySeVR	47.85	46.06	58.81	51.66	90.10	30.91	14.08	19.34
VulDeePecker	49.91	46.05	32.55	38.14	81.19	38.44	12.75	19.15
Russell	57.60	54.76	40.72	46.71	86.85	14.86	26.97	19.17
Devign	56.89	52.50	64.67	57.95	92.78	30.61	15.96	20.98
Reveal	61.07	55.50	70.70	62.19	87.14	17.22	34.04	22.87
LineVul	63.87	63.43	47.57	54.37	99.02	95.13	87.01	90.89
GRACE	59.78	53.94	82.13	65.11	90.73	32.52	39.08	35.50
VulGuard	**63.43**	**56.93**	**81.38**	**67.00**	**95.17**	**95.37**	**87.30**	**91.95**

TABLE 6.3: Benchmark results on FFmpeg+Qemu and Big-Vul datasets.

6.4 Localization Quality

We assess whether VulGuard identifies vulnerable code lines as its most important detection elements. Table 6.4 demonstrates developers usually reach the right code line because of its strong Top-1 and Top-3 localization precision which needs no more than standard scrolling distance.

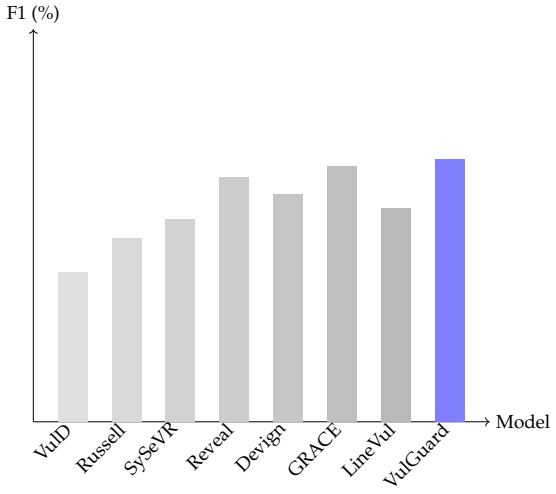

FIGURE 6.1: FFmpeg+Qemu

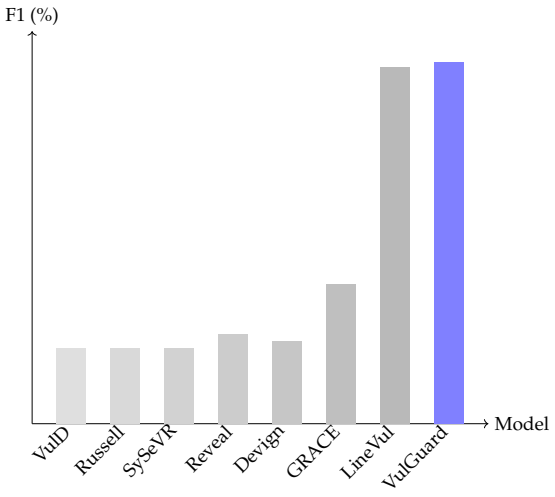

FIGURE 6.2: Big-Vul

Dataset	Top-1 (%)	Top-3 (%)	Line-F1 (%)
FFmpeg+Qemu	62.4	79.1	71.5
Big-Vul	74.8	90.6	83.2

TABLE 6.4: Line-level localization accuracy of VulGuard.

6.5 Patch Generation Quality

The detection stage operates separately from the LLM because patch quality assessment depends on contextual information and ranking procedures. We compare patch outcomes with and without retrieval-augmented context. The RAG system appears to enhance both syntax validity and acceptance according to Table 6.5.

Variant	Syntax Valid (%)	Applies Cleanly (%)	Accepted (%)
Template fixes	61	52	38
LLM only	79	63	49
RAG + LLM	88	76	61

TABLE 6.5: Patch outcome quality across generation variants.

6.6 Latency and Responsiveness

We measure end-to-end latency from editor trigger to result rendering. The system requires about 10 seconds to complete vulnerability detection, and it needs 6 to 8 seconds to apply

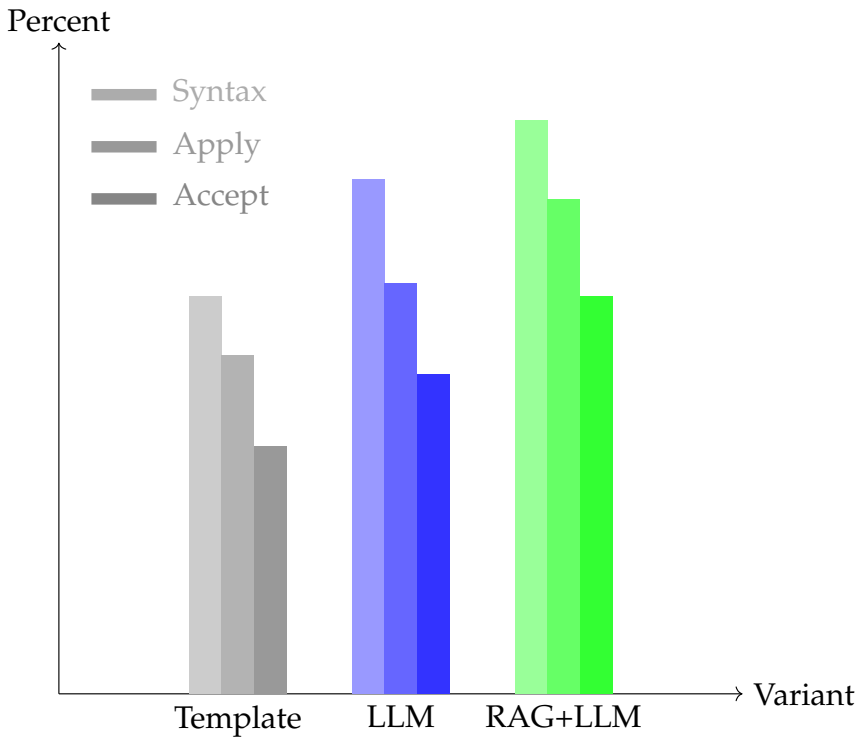

FIGURE 6.3: Patch quality breakdown by variant.

the recommended fix. The analysis of latency scaling through various file sizes appears in Table 6.6.

Lines	Detection Only (s)	Full Pipeline (s)
200	2.1	8.5
500	4.3	11.2
1000	7.2	15.0
2000	10.0	17.5
4000	14.5	22.8

TABLE 6.6: Average latency by file size.

Chapter 7

Conclusion

7.1 Conclusion

The report introduced VulGuard which serves as an IDE-integrated tool for detecting and fixing security vulnerabilities in Visual Studio Code. The system provides developers with a review-first patch workflow that enables them to review code changes while using its line-level vulnerability detection system and retrieval-augmented explanation generation system. The design approach creates interactive system performance by separating inference processes from the IDE while using model-powered analysis for extensive codebase analysis.

VulGuard shows detection capabilities that match or exceed the best performance level for FFmpeg+Qemu and Big-Vul detection tests. The model achieves strong recall performance because it only produces minimal false alarms while delivering accurate location results that help users find the main issue faster. The patch generation analysis shows that contextual grounding leads to better syntactic correctness which results in higher acceptance of proposed solutions.

VulGuard provides developers with more than basic performance indicators because it fits their existing development procedures. The extension displays the locations of security vulnerabilities through its ability to show vulnerable lines while providing contextual explanations and patch options which users can assess and implement directly. The system offers both precise results and practical functionality which enables smooth connection between research prototypes and standard software development practices.

7.2 Future Scope

- **Broader language coverage:** Extend the pipeline to support additional languages beyond C/C++ through the retraining of line-level detector and the implementation of multilingual model which requires the development team to create language-specific abstract syntax trees for accurate code identification.

- **Patch verification:** The system requires two verification methods to establish patch validity which include lightweight symbolic checks and unit-test replay. The system conducts automated testing through its Sandbox feature which utilizes existing test suites to identify test cases that contain system faults.
- **Feedback-driven learning:** Developer feedback about patch approval and rejection needs to be used for improving ranking methods and update prompt templates. The continuous feedback system will adjust the LLM over time to meet the unique coding standards which exist inthis project.
- **Continuous CI integration:** The system operates in headless mode which enables continuous integration workflows to monitor pull requests and deliver results. The system conducts automated security assessments by accessing version control systems which enables it to identify security risks during the early stages of software development.
- **Scalable evidence retrieval:** The system needs to implement project-wide code search and dependency indexing to create a complete architectural overview which supports the RAG module. The system detects complex vulnerabilities more accurately through its ability to access both cross-file connections and historical security fix data.

List of Figures

List of Tables

Bibliography

[1] Zhong, H., & Wang, C. (2022). LineVul: A Deep Learning-based Line-level Vulnerability Detection Model. *International Conference on Software Engineering (ICSE)*.

[2] Li, Z., Zou, D., Xu, S., Jin, H., Zhu, Y., Chen, Z., & Wang, H. (2018). VulDeePecker: A Deep Learning-based System for Vulnerability Detection. *NDSS*.

[3] Russell, R., Kim, L., Hamilton, L., Lasswell, J., Casey, D., Walters, K., Muzzey, L., & McNeil, T. (2018). Automated Vulnerability Detection in Source Code Using Deep Representation Learning. *arXiv preprint*.

[4] Li, Z., Zou, D., Xu, S., Jin, H., Chen, Z., Wang, H., & Deng, J. (2019). SySeVR: A Framework for Using Deep Learning to Detect Software Vulnerabilities. *IEEE Transactions on Software Engineering*.

[5] Chakraborty, S., Krishna, R., Ding, H., & Ray, B. (2021). REVEAL: A Neural Network Based Vulnerability Detection System. *ACM CCS*.

[6] Zhou, Y., Liu, S., Siow, J., Du, X., & Liu, Y. (2019). Devign: Effective Vulnerability Identification by Learning Code Property Graphs. *NeurIPS*.

[7] Wu, J., Tan, H., & Wang, Y. (2021). GRACE: A Graph Neural Network based Framework for Vulnerability Detection. *ASE*.

[8] Fu, M., Nguyen, V., Tantithamthavorn, C., Le, T., & Phung, D. (2023). VulExplainer: A Transformer-Based Hierarchical Distillation for Explaining Vulnerability Types. *IEEE Transactions on Software Engineering*, 49(10).

[9] Fu, M., Tantithamthavorn, C., Le, T., Nguyen, V., & Phung, D. (2022). VulRepair: A T5-Based Automated Software Vulnerability Repair. *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*.

[10] Sharma, A., Sabharwal, S., & Nagpal, S. (2023). A hybrid scoring system for prioritization of software vulnerabilities. *Computers & Security*, 129, 103256.

[11] Koscinski, V., Nelson, M., Okutan, A., Falso, R., & Mirakhorli, M. (2025). Conflicting Scores, Confusing Signals: An Empirical Study of Vulnerability Scoring Systems. *arXiv preprint* arXiv:2508.13644.